HDOC Day-51

Finding the Next Greater Element in an Array Using Functions and Pointers in C

Aim

To read a variable-length integer array and use a separate function named **next_greater()**, pointers, and nested loops to find the nearest strictly greater element to the right of each array element. Print -1 when no such element exists and separate results with commas.

Algorithm

1. Start the program and read the array size.
2. Validate that the size is positive.
3. Declare a variable-length array arr[size] and read its elements.
4. Pass the array and size to next_greater() using a pointer.
5. For each element, initialize next to -1.
6. Search elements to its right using an inner loop.
7. On finding the first strictly greater element, store it and break.
8. Print the result, adding a comma between adjacent outputs.
9. Repeat for every element and stop.

C Program

```
#include <stdio.h>

void next_greater(int *arr, int size);

int main()
{
    int size, i;

    printf("Enter the size of array: ");
    scanf("%d", &size);

    if (size <= 0)
    {
        printf("Invalid array size!\n");
        return 0;
    }

    int arr[size];
    printf("Enter %d array elements: ", size);

    for (i = 0; i < size; i++)
        scanf("%d", &arr[i]);

    next_greater(arr, size);
    return 0;
}

/* Function to find the next greater element */
void next_greater(int *arr, int size)
{
```

```
    int i, j, next;

    printf("\nNext Greater Elements: ");

    for (i = 0; i < size; i++)
    {
        next = -1;

        /* Search only to the right */
        for (j = i + 1; j < size; j++)
        {
            if (*(arr + j) > *(arr + i))
            {
                next = *(arr + j);
                break;
            }
        }

        printf("%d", next);
        if (i < size - 1)
            printf(", ");
    }
    printf("\n");
}
```

Explanation

The expression ***(arr + i)** accesses arr[i] through pointer arithmetic. The outer loop selects the current element, while the inner loop checks positions to its right. The first strictly greater element is chosen, not necessarily the smallest greater value.

Example

For the array **4 5 2 10 8**, the output is **5, 10, 10, -1, -1**.

Test Case Table

Case	Input array	Expected output
1	4 5 2 10 8	5, 10, 10, -1, -1
2	1 3 2 4	3, 4, 4, -1
3	5 4 3 2 1	-1, -1, -1, -1, -1
4	1 2 3 4 5	2, 3, 4, 5, -1
5	2 2 3 3 4	3, 3, 4, 4, -1
6	10 5 15 20	15, 15, 20, -1
7	7	-1
8	-3 -1 -2 4	-1, 4, 4, -1
9	size = 0	Invalid array size

Sample Outputs

```
Enter the size of array: 5
```

```
Enter 5 array elements: 4 5 2 10 8
Next Greater Elements: 5, 10, 10, -1, -1

Enter the size of array: 5
Enter 5 array elements: 5 4 3 2 1
Next Greater Elements: -1, -1, -1, -1, -1

Enter the size of array: 5
Enter 5 array elements: 2 2 3 3 4
Next Greater Elements: 3, 3, 4, 4, -1

Enter the size of array: 0
Invalid array size!
```

Result

The program uses a variable-length array, pointers, nested loops, and a user-defined function to determine the next strictly greater element to the right of every array element. Results are displayed in original order with commas, and -1 is printed whenever no greater element exists.